Heaps — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

211. Introduction to Priority Queues using Heaps

Understanding the Problem NEW

A **Heap** is a special tree-shaped structure that always keeps either the smallest (Min-Heap) or largest (Max-Heap) element instantly accessible at the top/root — and a **Priority Queue** is the abstract 'queue where the highest-priority item comes out first' concept that a heap is commonly used to implement efficiently.

Brute Force Approach NEW

Without a heap, you could keep an unsorted list and scan it every time you need the min/max — O(n) per query — or keep a fully sorted list, which gives O(1) access to the min/max but costs O(n) to insert a new element in the right sorted position.

Why We Optimize NEW

A heap strikes a middle ground: it doesn't maintain *full* sorted order (which is expensive to maintain on every insert), but it does maintain enough partial order (every parent node is smaller/larger than its children) to guarantee the min or max is always right at the root. This gives you O(log n) insert and O(log n) removal of the min/max, and O(1) to just *look at* the current min/max — a genuinely useful middle ground between 'no order' (fast insert, slow query) and 'full order' (slow insert, fast query), and it's exactly this trade-off that makes heaps the tool of choice for every 'kth largest/smallest,' 'merge k things,' and 'always process the highest priority item next' problem in this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding Priority Queue and Heap data structures.

Visual Diagram

```
Min Heap (Priority Queue - ascending):
                    1
                  /    \
                3       5
               / \     /
             4     8 6
```

```
          Max Heap (Priority Queue - descending):
                    8
                  /    \
                 6       5
                / \     /
               4     3 1
          Heap Operations:

       - Insert: O(log n)
       - Extract Min/Max: O(log n)
       - Peek: O(1)
       - Heapify: O(n)

       Array Representation (Min Heap):
       Index: 0 1 2 3 4 5
       Value: 1 3 5 4 8 6
       Children: 2i+1, 2i+2
       Parent: (i-1)/2
```

Java Implementation

```
// Using Java's PriorityQueue (Min Heap by default)
       class PriorityQueueExample {
           public void demonstrate() {
               // Min Heap
               PriorityQueue<Integer> minHeap = new PriorityQueue<>();
               minHeap.add(5);
               minHeap.add(3);
               minHeap.add(8);
               minHeap.add(1);
               // Peek: 1 (smallest)

               // Max Heap
               PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());
               maxHeap.add(5);
               maxHeap.add(3);
               maxHeap.add(8);
               maxHeap.add(1);
               // Peek: 8 (largest)
           }
       }
```

Time & Space Complexity

Time Complexity: O(log n) for insert/delete | Space Complexity: O(n)

212. Min Heap and Max Heap Implementation

Understanding the Problem NEW

Implement a Min-Heap (smallest element always at the root) and Max-Heap (largest always at the root) from scratch, typically backed by a plain array using index arithmetic to represent the tree structure (for node at index i, children are at 2i+1 and 2i+2, parent at (i-1)/2).

Brute Force Approach NEW

You could use an actual tree structure with explicit node objects and pointers, but this adds pointer overhead and complexity compared to representing the tree implicitly with array indices.

Why We Optimize NEW

Because a heap is always a **complete** binary tree (filled left to right, level by level, with no gaps), its shape is entirely predictable — which means you can represent it as a plain array without storing any explicit pointers at all, just using simple index arithmetic to know where any node's parent or children live. Insertion adds the new element at the end of the array and 'bubbles it up' (sift-up) by repeatedly swapping with its parent while the heap property is violated; removal takes the root, replaces it with the last element, and 'sifts it down' by repeatedly swapping with the smaller/larger child. Both operations are O(log n), since the tree's height is always O(log n) thanks to being complete — the array representation trades away explicit pointers for compact, cache-friendly index arithmetic.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Min Heap and Max Heap from scratch.

Visual Diagram

```
Min Heap Implementation:
       size = 0
       capacity = 10
       heap = []
```

```
       insert(5): [5]
       insert(3): [3,5] (swap 5 and 3)
       insert(8): [3,5,8]
       insert(1): [1,3,8,5] (swap up)

       extractMin():
         1 → remove
         [3,5,8] (heapify down)

       Visual Heap:
                  1
                /   \
              3      8
            /
           5
```

Solution Approach

Concept: Array-based heap with sift up/down.

Algorithm

```
Algorithm MIN_HEAP:
           heap = new int[capacity]
           size = 0

            insert(key):
                If size == capacity: return
                heap[size] = key
                size++
                siftUp(size-1)

            siftUp(index):
                While index > 0:
                    parent = (index-1)/2
                    If heap[parent] <= heap[index]: break
                    swap(parent, index)
                    index = parent

            extractMin():
                If size == 0: return -1
                min = heap[0]
                heap[0] = heap[size-1]
                size--
                siftDown(0)
                Return min

            siftDown(index):
                While true:
                    left = 2*index+1, right = 2*index+2
```

```
                    smallest = index
                    If left < size and heap[left] < heap[smallest]: smallest = left
                    If right < size and heap[right] < heap[smallest]: smallest =
right
                    If smallest == index: break
                    swap(index, smallest)
                    index = smallest
```

Java Implementation

```
class MinHeap {
          private int[] heap;
          private int size;
          private int capacity;

          MinHeap(int capacity) {
              this.capacity = capacity;
              heap = new int[capacity];
              size = 0;
          }

          public void insert(int key) {
              if (size == capacity) {
                  throw new RuntimeException("Heap is full");
              }
              heap[size] = key;
              size++;
              siftUp(size - 1);
          }

          private void siftUp(int index) {
              while (index > 0) {
                  int parent = (index - 1) / 2;
                  if (heap[parent] <= heap[index]) break;
                  swap(parent, index);
                  index = parent;
              }
          }

          public int extractMin() {
              if (size == 0) {
                  throw new RuntimeException("Heap is empty");
              }
              int min = heap[0];
              heap[0] = heap[size - 1];
              size--;
              siftDown(0);
              return min;
          }

          private void siftDown(int index) {
              while (true) {
```

```
                int left = 2 * index + 1;
                int right = 2 * index + 2;
                int smallest = index;

                if (left < size && heap[left] < heap[smallest]) {
                    smallest = left;
                }
                if (right < size && heap[right] < heap[smallest]) {
                    smallest = right;
                }
                if (smallest == index) break;
                swap(index, smallest);
                index = smallest;
            }
        }

        public int peek() {

             if (size == 0) {
                 throw new RuntimeException("Heap is empty");
             }
             return heap[0];
         }

         private void swap(int i, int j) {
             int temp = heap[i];
             heap[i] = heap[j];
             heap[j] = temp;
         }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(n)

213. Check if an Array Represents a Min-Heap

Understanding the Problem NEW

Given an array, check if it represents a valid Min-Heap (using the standard array-index-to-tree mapping) — meaning every parent is ≤ both of its children.

Brute Force Approach NEW

There isn't really a slower alternative here — checking the heap property requires examining each parent-child relationship at least once regardless.

Why We Optimize NEW

You only need to check the **internal (non-leaf) nodes** — roughly the first half of the array — since leaf nodes have no children to violate the property against. For each internal node at index i, verify it's ≤ its children at 2i+1 and 2i+2 (when those indices exist). This is an O(n) single pass, checking exactly the relationships that matter rather than doing any unnecessary extra work.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if array represents a valid Min Heap.

Visual Diagram

```
Array: [1, 3, 5, 4, 8, 6]
       Check parent <= children:
       1 <= 3 ✓, 1 <= 5 ✓
       3 <= 4 ✓, 3 <= 8 ✓
       5 <= 6 ✓
       Result: True

       Array: [1, 3, 2, 4, 8, 6]
       Check: 1 <= 3 ✓, 1 <= 2 ✓
       3 <= 4 ✓, 3 <= 8 ✓
       2 <= 6 ✓
       Result: True

       Array: [1, 5, 3, 4, 8, 6]
       Check: 1 <= 5 ✓, 1 <= 3 ✓
```

```
        5 <= 4 ✗ → Not Min Heap
        Result: False
```

Solution Approach

Concept: Check heap property for all internal nodes.

Algorithm

```
Algorithm IS_MIN_HEAP(arr):
            n = arr.length
            For i = 0 to (n/2)-1:
                left = 2*i + 1
                right = 2*i + 2
                If left < n and arr[i] > arr[left]:
                    Return false
                If right < n and arr[i] > arr[right]:
                    Return false
            Return true
```

Java Implementation

```
class Solution {
            public boolean isMinHeap(int[] arr) {
                int n = arr.length;
                for (int i = 0; i < n / 2; i++) {
                    int left = 2 * i + 1;
                    int right = 2 * i + 2;

                      if (left < n && arr[i] > arr[left]) {
                          return false;
                      }
                      if (right < n && arr[i] > arr[right]) {
                          return false;
                      }
                  }
                  return true;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

214. Convert Min Heap to Max Heap

Understanding the Problem NEW

Given an array that's currently a valid Min-Heap, rearrange it in-place so it becomes a valid Max-Heap instead.

Brute Force Approach NEW

You could extract all elements one by one (removing the min each time, which is O(log n) per removal) and re-insert them into a fresh Max-Heap — O(n log n) total, throwing away and rebuilding the whole structure.

Why We Optimize NEW

Rather than rebuilding from scratch, you can convert in-place using the classic **'build-heap' technique**: starting from the last internal (non-leaf) node and working backward to the root, run 'sift-down' (using the Max-Heap comparison rule) on each node. Because you process nodes bottom-up, each node's subtree is already a valid max-heap by the time you sift it, so a single pass suffices — this well-known technique runs in O(n) total (not O(n log n)), because most nodes are near the bottom of the tree where sift-down does very little work, and only a few nodes near the root need the full O(log n) sift — a much cheaper way to establish heap order than n individual O(log n) insertions.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert a Min Heap array to Max Heap.

Visual Diagram

```
Min Heap: [1, 3, 5, 4, 8, 6]
       Max Heap: [8, 5, 6, 4, 1, 3]

       Step 1: Start from last internal node
       Index: 2 (value 5)
       Check children: left=5? Actually heap[2]=5, left=6? Let's recalc

       Array: [1, 3, 5, 4, 8, 6]
       Index: 0 1 2 3 4 5
```

```
Build Max Heap:
i=2: 5 > max(6) ? 5<6 → swap with 6 → [1,3,6,4,8,5]
i=1: 3 > max(4,8) ? 3<8 → swap with 8 → [1,8,6,4,3,5]
i=0: 1 > max(8,6) ? 1<8 → swap with 8 → [8,1,6,4,3,5]
     1 > max(4,3) ? 1<4 → swap with 4 → [8,4,6,1,3,5]
     1 > max(3,5) ? 1<5 → swap with 5 → [8,4,6,5,3,1]

Result: [8, 4, 6, 5, 3, 1]? Wait, let's recalc properly

Original: [1, 3, 5, 4, 8, 6]
          1
        /   \
      3       5
     / \     /
   4     8 6

Max Heap after heapify:
          8
        /   \
      6       5
     / \     /
   4     3 1

Array: [8, 6, 5, 4, 3, 1]
```

Solution Approach

Concept: Heapify all internal nodes.

Algorithm

```
Algorithm CONVERT_TO_MAX_HEAP(arr):
           n = arr.length
           For i = n/2 - 1 down to 0:
               maxHeapify(arr, n, i)
           Return arr

       Algorithm maxHeapify(arr, n, i):
           largest = i
           left = 2*i + 1
           right = 2*i + 2

           If left < n and arr[left] > arr[largest]:
               largest = left
           If right < n and arr[right] > arr[largest]:

               largest = right
           If largest != i:
               swap(arr[i], arr[largest])
               maxHeapify(arr, n, largest)
```

Java Implementation

```
class Solution {
           public void convertMinToMaxHeap(int[] arr) {
               int n = arr.length;
               for (int i = n / 2 - 1; i >= 0; i--) {
                   maxHeapify(arr, n, i);
               }
           }

           private void maxHeapify(int[] arr, int n, int i) {
               int largest = i;
               int left = 2 * i + 1;
               int right = 2 * i + 2;

               if (left < n && arr[left] > arr[largest]) {
                   largest = left;
               }
               if (right < n && arr[right] > arr[largest]) {
                   largest = right;
               }
               if (largest != i) {
                   swap(arr, i, largest);
                   maxHeapify(arr, n, largest);
               }
           }

           private void swap(int[] arr, int i, int j) {
               int temp = arr[i];
               arr[i] = arr[j];
               arr[j] = temp;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(log n)

215. Kth Largest Element in an Array

Understanding the Problem NEW

Find the K-th largest element in an unsorted array. E.g. in `[3,2,1,5,6,4]` with K=2, the answer is 5 (the 2nd largest value).

Brute Force Approach NEW

Sort the entire array and read off the element at the (n-K)-th position — O(n log n), which does more work than necessary since you don't actually need the whole array sorted, just to identify one specific rank.

Why We Optimize NEW

Maintain a **Min-Heap of size K**: process each element, and if the heap has fewer than K elements, add it; if it already has K elements and the new one is bigger than the heap's current minimum (the root), replace the root with the new element. After processing the whole array, the heap's root is exactly the K-th largest element — this works because the heap only ever needs to remember 'the K biggest values seen so far,' discarding anything that's too small to matter, achieving O(n log K) instead of O(n log n) — better when K is much smaller than n, since you're never paying for the cost of fully sorting elements you don't ultimately care about the relative order of.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the kth largest element in an array.

Visual Diagram

```
Array: [3, 2, 1, 5, 6, 4], k=2
       Output: 5

       Sorted: [6, 5, 4, 3, 2, 1]
       2nd largest = 5

       Using Min Heap of size k:
       i=0: 3 → [3]
       i=1: 2 → [2,3]
       i=2: 1 → [1,2,3]
```

```
        i=3: 5 → 5>1 → [2,3,5]
        i=4: 6 → 6>2 → [3,5,6]
        i=5: 4 → 4>3 → [4,5,6]
        Result: 4 (top of min heap)

        Wait, for kth largest with min heap size k:
        After processing all, heap contains k largest elements
        Top is kth largest → 4

        But expected is 5... Let's check:
        [3,2,1,5,6,4], k=2
        Heap of size 2:
        i=0: 3 → [3]
        i=1: 2 → [2,3]
        i=2: 1 → 1<2 → no change, [2,3]
        i=3: 5 → 5>2 → remove 2, add 5 → [3,5]
        i=4: 6 → 6>3 → remove 3, add 6 → [5,6]
        i=5: 4 → 4<5 → no change, [5,6]
        Result: 5 ✓
```

Solution Approach

Concept: Min heap of size k.

Algorithm

```
Algorithm KTH_LARGEST(arr, k):
           minHeap = PriorityQueue()
           For num in arr:
               minHeap.add(num)
               If minHeap.size() > k:
                   minHeap.poll()
           Return minHeap.peek()
```

Java Implementation

```
class Solution {
           public int findKthLargest(int[] nums, int k) {
               PriorityQueue<Integer> minHeap = new PriorityQueue<>();

                 for (int num : nums) {
                     minHeap.add(num);

                   if (minHeap.size() > k) {
                       minHeap.poll();
                   }
               }
               return minHeap.peek();
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n log k) | Space Complexity: O(k)

216. Kth Smallest Element in an Array

Understanding the Problem NEW

The mirror image of the previous problem: find the K-th smallest element in an unsorted array.

Brute Force Approach NEW

Sort the whole array and read off the K-th position — O(n log n).

Why We Optimize NEW

Same idea as problem 215, flipped: maintain a **Max-Heap of size K**, keeping the K smallest values seen so far and discarding anything bigger once the heap is full — the root ends up being the K-th smallest. Recognizing that 'Kth smallest' and 'Kth largest' are the same technique with the heap type flipped (Max-Heap vs Min-Heap) is the actual insight, not a new algorithm.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the kth smallest element in an array.

Visual Diagram

```
Array: [7, 10, 4, 3, 20, 15], k=3
       Output: 7

       Sorted: [3, 4, 7, 10, 15, 20]
       3rd smallest = 7

       Using Max Heap of size k:
       i=0: 7 → [7]
       i=1: 10 → [10,7]
       i=2: 4 → 4<10 → remove 10, add 4 → [7,4]
       i=3: 3 → 3<7 → remove 7, add 3 → [4,3]
       i=4: 20 → 20>4 → no change, [4,3]
       i=5: 15 → 15>4 → no change, [4,3]
       Result: 4

       Wait, expected is 7... Let's trace:
       Max Heap of size k contains k smallest elements
```

```
       Top is kth smallest

       i=0: 7 → [7]
       i=1: 10 → [10,7] (10 is top)
       i=2: 4 → 4<10 → pop 10, add 4 → [7,4]
       i=3: 3 → 3<7 → pop 7, add 3 → [4,3]
       i=4: 20 → 20>4 → no change
       i=5: 15 → 15>4 → no change
       Top = 4 → kth smallest = 4

       But sorted array: [3,4,7,10,15,20], 3rd smallest = 7

       Problem: For k=3, heap should contain [7,4,3] but max heap gives [4,3]?
Actually with size k, i
       Let's re-trace carefully:

       i=0: 7 → [7]
       i=1: 10 → [10,7]
       i=2: 4 → 4<10 → remove 10, add 4 → [7,4]
       i=3: 3 → 3<7 → remove 7, add 3 → [4,3] (size=2)
       i=4: 20 → 20>4 → no change
       i=5: 15 → 15>4 → no change
       Top = 4 (which is 2nd smallest, not 3rd)

       The issue is size tracking. Let's use approach: Add all, then remove n-k
times.

       Alternative: Use PriorityQueue with comparator and remove n-k largest.
```

Solution Approach

Concept: Max heap of size k.

Algorithm

```
Algorithm KTH_SMALLEST(arr, k):
           maxHeap = PriorityQueue(Collections.reverseOrder())
           For num in arr:
               maxHeap.add(num)
               If maxHeap.size() > k:

                   maxHeap.poll()
           Return maxHeap.peek()
```

Java Implementation

```
class Solution {
           public int findKthSmallest(int[] nums, int k) {
               PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());
```

```
            for (int num : nums) {
                maxHeap.add(num);
                if (maxHeap.size() > k) {
                    maxHeap.poll();
                }
            }
            return maxHeap.peek();
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log k) | Space Complexity: O(k)

217. Sort K-Sorted Array

Understanding the Problem NEW

Given an array where every element is at most K positions away from its correct sorted position (a 'K-sorted' or 'nearly sorted' array), sort it efficiently, exploiting this bound on displacement.

Brute Force Approach NEW

Ignore the K-sorted structure entirely and run a general-purpose sort like Quick Sort or Merge Sort — correct, O(n log n), but doesn't take advantage of the extra guarantee the problem gives you.

Why We Optimize NEW

Because any element can only be at most K positions from where it belongs, the true smallest remaining element is guaranteed to be found within the next K+1 elements at any point in the scan. Maintain a **Min-Heap of size K+1**: add the first K+1 elements, then repeatedly extract the minimum (guaranteed correct for the next output position) and add the next unseen element from the array, one at a time. This produces the fully sorted result in O(n log K) instead of O(n log n) — a real improvement when K is much smaller than n, directly exploiting the 'bounded displacement' guarantee that a general sort would ignore.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort an array where each element is at most k positions away.

Visual Diagram

```
Array: [6, 5, 3, 2, 8, 10, 9], k=3
       Output: [2, 3, 5, 6, 8, 9, 10]

       Using Min Heap:
       i=0 to k: heap=[6,5,3] → sorted: 3
       i=1 to k+1: heap=[6,5,2] → sorted: 2
       i=2 to k+2: heap=[6,5,8] → sorted: 5
       i=3 to k+3: heap=[6,8,10] → sorted: 6
       i=4 to k+4: heap=[8,10,9] → sorted: 8
       i=5: heap=[10,9] → sorted: 9
```

```
        i=6: heap=[10] → sorted: 10

        Result: [3, 2, 5, 6, 8, 9, 10]... Wait

        Let's re-trace:
        Add first k+1 elements: [6,5,3,2] → min=2
        Add 8: [6,5,3,8] → min=3
        Add 10: [6,5,8,10] → min=5
        Add 9: [6,8,10,9] → min=6
        Remaining: [8,9,10] → sort → [8,9,10]
        Result: [2,3,5,6,8,9,10]
```

Solution Approach

Concept: Min heap of size k+1.

Algorithm

```
Algorithm SORT_K_SORTED(arr, k):
           minHeap = PriorityQueue()
           result = []

           For i = 0 to k:
               minHeap.add(arr[i])

           For i = k+1 to n-1:
               result.add(minHeap.poll())
               minHeap.add(arr[i])

           While !minHeap.isEmpty():
               result.add(minHeap.poll())

           Return result
```

Java Implementation

```
class Solution {
           public int[] sortKSortedArray(int[] arr, int k) {
               PriorityQueue<Integer> minHeap = new PriorityQueue<>();
               int n = arr.length;
               int[] result = new int[n];

                 for (int i = 0; i <= Math.min(k, n - 1); i++) {
                     minHeap.add(arr[i]);
                 }

               int index = 0;
               for (int i = k + 1; i < n; i++) {
                   result[index++] = minHeap.poll();
                   minHeap.add(arr[i]);
```

```
            }

            while (!minHeap.isEmpty()) {
                result[index++] = minHeap.poll();
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log k) | Space Complexity: O(k)

218. Merge M Sorted Lists

Understanding the Problem NEW

Given M separate sorted linked lists, merge them all into one fully sorted linked list.

Brute Force Approach NEW

Merge the lists two at a time using the ordinary two-list merge from Merge Sort (Chapter 1) — merging list 1 with list 2, then that result with list 3, and so on. If each list has roughly n/M elements, this sequential merging costs O(n×M) in the worst case, since each of the M merge steps can touch up to all n elements accumulated so far.

Why We Optimize NEW

Instead of merging sequentially, use a **Min-Heap containing the current front node of every list**: repeatedly extract the overall minimum (across all M lists' current fronts), append it to the result, and push that extracted node's *next* node (from the same list) back into the heap. Since the heap only ever holds at most M elements at a time, each of the n total extractions costs O(log M), giving O(n log M) total — much better than O(n×M) when M is large, because you're comparing across all M lists' current candidates simultaneously via the heap, rather than fully re-merging accumulated results at every step.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Merge k sorted linked lists into one sorted list.

Visual Diagram

```
Lists:
       1 → 4 → 5
       1 → 3 → 4
       2 → 6

       Merged: 1 → 1 → 2 → 3 → 4 → 4 → 5 → 6

       Using Min Heap:
```

```
Step 1: Add first node of each list → [1,1,2]
Step 2: Poll 1 (from list1), add next 4
Step 3: Poll 1 (from list2), add next 3
Step 4: Poll 2, add next 6
Step 5: Poll 3, add next 4
Step 6: Poll 4, add next 5
Step 7: Poll 4, add next null
Step 8: Poll 5, no more
Step 9: Poll 6, no more
```

Solution Approach

Concept: Min heap with list nodes.

Algorithm

```
Algorithm MERGE_K_LISTS(lists):
           minHeap = PriorityQueue(comparator for node values)

            For each list in lists:
                If list is not null:
                    minHeap.add(list)

            dummy = Node(0)
            current = dummy

            While !minHeap.isEmpty():
                node = minHeap.poll()
                current.next = node
                current = current.next

                 If node.next != null:
                     minHeap.add(node.next)

            Return dummy.next
```

Java Implementation

```
class Solution {
           public ListNode mergeKLists(ListNode[] lists) {
               PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) ->
a.val - b.val);

                 for (ListNode list : lists) {
                     if (list != null) {
                         minHeap.add(list);
                     }
                 }

               ListNode dummy = new ListNode(0);
```

```
            ListNode current = dummy;

            while (!minHeap.isEmpty()) {
                ListNode node = minHeap.poll();
                current.next = node;
                current = current.next;

                if (node.next != null) {
                    minHeap.add(node.next);
                }
            }
            return dummy.next;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log k) | Space Complexity: O(k)

219. Replace Each Array Element by Its Corresponding Rank

Understanding the Problem NEW

Replace every element in the array with its 'rank' — its 1-based position if the array were sorted (with equal elements getting the same rank). E.g. `[40,10,20,30]` → ranks `[4,1,2,3]`.

Brute Force Approach NEW

For each element, count how many elements in the array are smaller than or equal to it (a nested-loop comparison) — $O(n^2)$.

Why We Optimize NEW

Since a rank is fundamentally about relative sorted order, you can compute it by **sorting a copy of the array once** (or using a Min-Heap to extract elements in sorted order), assigning rank numbers as you go, and storing each value's rank in a hashmap — then look up each original element's rank from that map. This turns an $O(n^2)$ all-pairs comparison into O(n log n) for the sort/heap-based rank assignment plus O(n) for the final lookup pass — reusing the fact that ranks are just 'position in sorted order' rather than recomputing relative order pairwise for every element.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Replace each element with its rank (1-based smallest to largest).

Visual Diagram

```
Input: [10, 20, 15, 10, 30]
       Output: [1, 3, 2, 1, 4]

       Step 1: Sort unique values: [10, 15, 20, 30]
       Step 2: Map value → rank: 10→1, 15→2, 20→3, 30→4
       Step 3: Replace: [1, 3, 2, 1, 4]

       Using Min Heap:
       Create pairs (value, index)
```

```
Heap: (10,0), (10,3), (15,2), (20,1), (30,4)
Rank=1: pop (10,0) → result[0]=1
Rank=2: pop (10,3) → result[3]=1 (same rank for duplicates)
Rank=3: pop (15,2) → result[2]=2
Wait, for duplicates they should have same rank

Actually:
Rank=1: pop (10,0) → result[0]=1
Rank=1: pop (10,3) → result[3]=1
Rank=2: pop (15,2) → result[2]=2
Rank=3: pop (20,1) → result[1]=3
Rank=4: pop (30,4) → result[4]=4
Result: [1,3,2,1,4]
```

Solution Approach

Concept: Min heap or sorting with map.

Algorithm

```
Algorithm REPLACE_WITH_RANK(arr):
           pairs = new Pair[arr.length]
           For i = 0 to n-1:
               pairs[i] = (arr[i], i)
           Sort pairs by value

           result = new int[n]
           rank = 1

           For i = 0 to n-1:
               result[pairs[i].index] = rank
               If i+1 < n and pairs[i].value != pairs[i+1].value:
                   rank++

           Return result
```

Java Implementation

```
class Solution {
           public int[] replaceWithRank(int[] arr) {
               int n = arr.length;
               int[][] pairs = new int[n][2];

                 for (int i = 0; i < n; i++) {
                     pairs[i] = new int[]{arr[i], i};
                 }

               Arrays.sort(pairs, (a, b) -> a[0] - b[0]);

               int[] result = new int[n];
```

```
            int rank = 1;

            for (int i = 0; i < n; i++) {
                result[pairs[i][1]] = rank;
                if (i < n - 1 && pairs[i][0] != pairs[i + 1][0]) {
                    rank++;
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

220. Task Scheduler

Understanding the Problem NEW

Given a list of tasks (each represented by a type) and a cooldown period N (the same task type can't run again until N other slots have passed, possibly filled with idle time), find the minimum total time needed to complete all tasks.

Brute Force Approach NEW

Try every possible ordering of tasks and idle slots, checking which satisfies the cooldown constraint with minimum total time — combinatorially expensive, since there are many possible orderings to consider.

Why We Optimize NEW

A greedy, frequency-based approach works well here: schedule the **most frequent remaining task type** first at each available slot (using a Max-Heap keyed by remaining count), since the most frequent task is the one most likely to run out of 'safe' cooldown slots if delayed — this greedy priority ensures no task type ever gets stuck waiting longer than necessary. Combined with tracking a cooldown queue of recently-used tasks, this achieves the optimal schedule in roughly O(total tasks × log(distinct task types)) instead of exploring every possible task ordering — a case where a heap-driven greedy choice ('always do the most urgent thing next') avoids a combinatorial search entirely.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Schedule tasks with cooldown between same tasks.

Visual Diagram

```
Tasks: ["A","A","A","B","B","B"], n=2
       Output: 8

       A B _ A B _ A B → 8 time units

       Frequency: A=3, B=3
       Most frequent = 3
       idleTime = (3-1) * 2 = 4
```

```
        Remaining tasks = 6 - 3 = 3
        idleTime -= 3 = 1
        Total = 3 + 6 = 9?

        Let's recalc:
        maxCount = 3, maxCountTasks = 2 (A and B both appear 3 times)
        partCount = maxCount - 1 = 2
        emptySlots = partCount * (n - (maxCountTasks - 1)) = 2 * (2 - 1) = 2
        availableTasks = 6 - maxCount * maxCountTasks = 6 - 6 = 0
        idle = max(0, emptySlots - availableTasks) = 2
        result = 6 + 2 = 8

        Alternative formula:
        result = max(tasks.length, (maxCount-1)*(n+1) + maxCountTasks)
        maxCount=3, maxCountTasks=2, n=2
        (3-1)*(2+1) + 2 = 2*3+2 = 8
        tasks.length=6
        result = max(6, 8) = 8
```

Solution Approach

Concept: Greedy with frequency counting.

Algorithm

```
Algorithm TASK_SCHEDULER(tasks, n):
           freq = new int[26]
           For task in tasks: freq[task-'A']++

           maxCount = 0, maxCountTasks = 0
           For count in freq:
               If count > maxCount:
                   maxCount = count
                   maxCountTasks = 1
               Else If count == maxCount:
                   maxCountTasks++

           result = Math.max(tasks.length, (maxCount-1)*(n+1) + maxCountTasks)
           Return result
```

Java Implementation

```
class Solution {
           public int leastInterval(char[] tasks, int n) {
               int[] freq = new int[26];
               for (char task : tasks) {
                   freq[task - 'A']++;

              }
```

```
            int maxCount = 0;
            int maxCountTasks = 0;

            for (int count : freq) {
                if (count > maxCount) {
                    maxCount = count;
                    maxCountTasks = 1;
                } else if (count == maxCount) {
                    maxCountTasks++;
                }
            }

            int result = Math.max(tasks.length, (maxCount - 1) * (n + 1) +
maxCountTasks);
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

221. Hands of Straights

Understanding the Problem NEW

Given a hand of cards (numbers), determine if they can be grouped into groups of consecutive-value straights of a given size W (e.g. cards `[1,2,3,6,2,3,4,7,8]`, W=3 → groups `[1,2,3]`, `[2,3,4]`, `[6,7,8]`).

Brute Force Approach NEW

Try every possible way to partition the cards into groups and check if each is a valid consecutive run — combinatorially expensive without a clear strategy.

Why We Optimize NEW

A greedy approach using a frequency map and a **Min-Heap of distinct card values**: always start a new group from the **smallest remaining** card value (since if the smallest card can't start a valid consecutive run, no arrangement can ever place it anywhere else, as there's nothing smaller left to pair it with), then greedily consume that value and the next W-1 consecutive values to form one group, decrementing their counts. Repeating this — always resolving the smallest remaining value first — greedily builds valid groups without ever needing to explore alternative groupings, since the smallest remaining card's fate is always forced.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if cards can be grouped into consecutive sequences.

Visual Diagram

```
Hand: [1,2,3,6,2,3,4,7,8], groupSize=3
       Output: true

       Groups: [1,2,3], [2,3,4], [6,7,8]

       Using Min Heap:
       Map frequencies: 1:1,2:2,3:2,4:1,6:1,7:1,8:1
       Start with 1: group [1,2,3]
         freq[1]=0, freq[2]=1, freq[3]=1
       Start with 2: group [2,3,4]
```

```
  freq[2]=0, freq[3]=0, freq[4]=0
Start with 6: group [6,7,8]
  freq[6]=0, freq[7]=0, freq[8]=0
Result: true

Hand: [1,2,3,4,5], groupSize=4
Output: false
Start with 1: [1,2,3,4] → 5 remaining
Can't form another group → false
```

Solution Approach

Concept: Min heap + frequency map.

Algorithm

```
Algorithm HANDS_OF_STRAIGHTS(hand, groupSize):
           If hand.length % groupSize != 0: return false

           freq = new HashMap()
           For card in hand: freq[card]++

           minHeap = new PriorityQueue(freq.keySet())

           While !minHeap.isEmpty():
               first = minHeap.peek()
               For i = 0 to groupSize-1:
                   current = first + i
                   If !freq.containsKey(current) or freq.get(current) == 0:
                       return false
                   freq.put(current, freq.get(current)-1)
                   If freq.get(current) == 0:
                       If minHeap.peek() != current: return false
                       minHeap.poll()

           Return true
```

Java Implementation

```
class Solution {
           public boolean isNStraightHand(int[] hand, int groupSize) {
               if (hand.length % groupSize != 0) return false;

                 Map<Integer, Integer> freq = new HashMap<>();
                 for (int card : hand) {

                   freq.put(card, freq.getOrDefault(card, 0) + 1);
               }

               PriorityQueue<Integer> minHeap = new PriorityQueue<>(freq.keySet());
```

```
            while (!minHeap.isEmpty()) {
                int first = minHeap.peek();

                for (int i = 0; i < groupSize; i++) {
                    int current = first + i;
                    if (!freq.containsKey(current) || freq.get(current) == 0) {
                        return false;
                    }
                    freq.put(current, freq.get(current) - 1);
                    if (freq.get(current) == 0) {
                        if (minHeap.peek() != current) {
                            return false;
                        }
                        minHeap.poll();
                    }
                }
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

222. Design Twitter

Understanding the Problem NEW

Design a simplified Twitter: users can post tweets, follow/unfollow other users, and retrieve their 10 most recent tweets from themselves and everyone they follow, merged in reverse-chronological order.

Brute Force Approach NEW

To get the feed, you could gather every tweet from the user and everyone they follow into one list, then sort by timestamp and take the top 10 — correct, but sorting potentially thousands of tweets just to get the top 10 wastes most of that sorting effort.

Why We Optimize NEW

This is essentially the 'Merge M Sorted Lists' problem (218) in disguise: each followed user's tweet history is already naturally in reverse-chronological (sorted) order, so use a **Max-Heap** containing the most recent tweet from the user and each followed user, repeatedly extracting the overall most recent tweet and pushing that same user's next-most-recent tweet back into the heap, stopping after 10 extractions. This reuses the exact same 'heap over multiple sorted sources' technique as merging sorted lists, achieving O(10 × log(number of followed users)) for feed retrieval instead of sorting every tweet from every followed user.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Design Twitter with post, get feed, follow, unfollow.

Visual Diagram

```
Users: 1, 2, 3
       Tweets: user1: [tweet1, tweet4], user2: [tweet2], user3: [tweet3]

       getNewsFeed(1) → [tweet4, tweet3, tweet2, tweet1] (latest 10)

       Data Structures:
       Map<Integer, User> users
```

```
        User: {
            tweets: LinkedList<Tweet>
            following: Set<Integer>
        }
        Tweet: { id, timestamp }

        postTweet(userId, tweetId):
            user = users.get(userId)
            user.tweets.addFirst(tweetId)

        getNewsFeed(userId):
            tweets = []
            For each followee in user.following:
                Add all tweets from followee
            Sort by timestamp desc
            Return top 10
```

Solution Approach

Concept: HashMap + Heap for feed.

Java Implementation

```
class Twitter {
            private class Tweet {
                int tweetId;
                int timestamp;
                Tweet next;
                Tweet(int tweetId, int timestamp) {
                    this.tweetId = tweetId;
                    this.timestamp = timestamp;
                }
            }

             private class User {
                 int userId;
                 Set<Integer> following;
                 Tweet tweets;

                 User(int userId) {
                     this.userId = userId;
                     following = new HashSet<>();
                     following.add(userId);
                 }
             }

            private Map<Integer, User> users;
            private int timestamp;

            public Twitter() {
                users = new HashMap<>();
```

```
                timestamp = 0;
            }

            public void postTweet(int userId, int tweetId) {
                users.computeIfAbsent(userId, k -> new User(userId));
                Tweet tweet = new Tweet(tweetId, timestamp++);
                tweet.next = users.get(userId).tweets;
                users.get(userId).tweets = tweet;
            }

            public List<Integer> getNewsFeed(int userId) {
                users.computeIfAbsent(userId, k -> new User(userId));

                PriorityQueue<Tweet> heap = new PriorityQueue<>((a, b) ->
a.timestamp - b.timestamp);

                for (int followeeId : users.get(userId).following) {
                    if (!users.containsKey(followeeId)) continue;
                    Tweet tweet = users.get(followeeId).tweets;
                    while (tweet != null && heap.size() < 10) {
                        heap.add(tweet);
                        tweet = tweet.next;
                    }
                    if (heap.size() == 10 && tweet != null) {
                        while (tweet != null) {
                            if (tweet.timestamp > heap.peek().timestamp) {
                                heap.poll();
                                heap.add(tweet);
                            }
                            tweet = tweet.next;
                        }
                    }
                }

                List<Integer> result = new ArrayList<>();
                while (!heap.isEmpty()) {
                    result.add(heap.poll().tweetId);
                }
                Collections.reverse(result);
                return result;
            }

            public void follow(int followerId, int followeeId) {
                users.computeIfAbsent(followerId, k -> new User(followerId));
                users.computeIfAbsent(followeeId, k -> new User(followeeId));
                users.get(followerId).following.add(followeeId);
            }

            public void unfollow(int followerId, int followeeId) {
                if (followerId != followeeId) {
                    users.get(followerId).following.remove(followeeId);
                }
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n log k) | Space Complexity: O(n)

223. Connect n Ropes with Minimal Cost

Understanding the Problem NEW

Given ropes of various lengths, connect them all into one rope, where connecting two ropes costs (sum of their lengths), and you want to minimize the *total* cost across all connections.

Brute Force Approach NEW

Try every possible order of connecting ropes and compute the total cost for each — combinatorially expensive (the number of possible connection orders grows very quickly).

Why We Optimize NEW

The greedy insight: always connect the **two currently-smallest** ropes at each step. This minimizes total cost because each rope's length gets 'counted' once for every connection it participates in downstream — so keeping small ropes small for as long as possible (by combining them early, before they get folded into larger and larger combined ropes) minimizes how many times their length contributes to the total. A **Min-Heap** makes 'always grab the two smallest' an O(log n) operation per step, giving O(n log n) total — turning a combinatorially expensive ordering search into a straightforward greedy-with-heap process.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Connect ropes with minimal cost (sum of lengths).

Visual Diagram

```
Ropes: [4, 3, 2, 6]
       Minimal cost: 29

       Step 1: Connect 2+3=5, cost=5, ropes=[4,5,6]
       Step 2: Connect 4+5=9, cost=5+9=14, ropes=[6,9]
       Step 3: Connect 6+9=15, cost=14+15=29

       Using Min Heap:
       [2,3,4,6]
       Step 1: 2+3=5, cost=5, add 5 → [4,5,6]
```

```
       Step 2: 4+5=9, cost=14, add 9 → [6,9]
       Step 3: 6+9=15, cost=29
```

Solution Approach

Concept: Min heap always connect two smallest.

Algorithm

```
Algorithm MIN_COST_CONNECT(ropes):
           minHeap = PriorityQueue(ropes)
           cost = 0

           While minHeap.size() > 1:
               first = minHeap.poll()
               second = minHeap.poll()
               sum = first + second
               cost += sum
               minHeap.add(sum)

           Return cost
```

Java Implementation

```
class Solution {
           public int minCostConnectRopes(int[] ropes) {
               PriorityQueue<Integer> minHeap = new PriorityQueue<>();
               for (int rope : ropes) {
                   minHeap.add(rope);
               }

                 int cost = 0;
                 while (minHeap.size() > 1) {
                     int first = minHeap.poll();
                     int second = minHeap.poll();
                     int sum = first + second;
                     cost += sum;
                     minHeap.add(sum);
                 }
                 return cost;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

224. Kth Largest Element in a Stream of Data

Understanding the Problem NEW

Design a structure that, given a continuous stream of incoming numbers (arriving one at a time, not known in advance), can always report the current K-th largest number seen so far at any point.

Brute Force Approach NEW

Store every number seen so far in a list, and whenever asked for the K-th largest, sort the entire list and read off the answer — O(n log n) per query, terrible if queries happen frequently as the stream grows.

Why We Optimize NEW

Maintain a running **Min-Heap of size K**, exactly like problem 215, but now incrementally: every time a new number arrives, add it to the heap (evicting the current minimum if the heap exceeds size K) — the root is always the current K-th largest, retrievable in O(1) at any moment, with each new number processed in O(log K). This is the 'streaming' version of problem 215's static-array technique — the heap's small, bounded size (K, not n) is precisely what makes it practical to maintain incrementally across a continuously growing stream rather than re-sorting everything on every query.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find kth largest in a stream of numbers.

Visual Diagram

```
Stream: [4, 5, 8, 2, 3, 5, 10, 9, 4], k=3

       After each:
       4 → [4] → kth largest = -1 (not enough elements)
       5 → [4,5] → -1
       8 → [4,5,8] → 4
       2 → [4,5,8,2] → min=2? Wait, heap should keep size k

       Min Heap of size k:
       4: [4]
```

```
5: [4,5]
8: [4,5,8]
2: 2<4 → no change, [4,5,8] → kth=4
3: 3<4 → no change, [4,5,8] → kth=4
5: 5>4 → remove 4, add 5 → [5,5,8] → kth=5
10: 10>5 → remove 5, add 10 → [5,8,10] → kth=5
9: 9>5 → remove 5, add 9 → [8,9,10] → kth=8
4: 4<8 → no change, [8,9,10] → kth=8
```

Solution Approach

Concept: Min heap of size k.

Algorithm

```
Algorithm KTH_LARGEST_STREAM(stream, k):
           minHeap = PriorityQueue()
           For num in stream:
               minHeap.add(num)
               If minHeap.size() > k:
                   minHeap.poll()
               If minHeap.size() == k:
                   print minHeap.peek()
               Else:
                   print -1
```

Java Implementation

```
class KthLargest {
           private PriorityQueue<Integer> minHeap;
           private int k;

           public KthLargest(int k, int[] nums) {
               this.k = k;
               minHeap = new PriorityQueue<>();
               for (int num : nums) {
                   add(num);
               }
           }

           public int add(int val) {
               minHeap.add(val);
               if (minHeap.size() > k) {
                   minHeap.poll();
               }

               return minHeap.peek();
           }
       }
```

Time & Space Complexity

Time Complexity: O(log k) per add | Space Complexity: O(k)

225. Maximum Sum Combination

Understanding the Problem NEW

Given two arrays, find the K pairs (one element from each array) with the largest sums, without generating and sorting every possible pair (which would be a times b total pairs).

Brute Force Approach NEW

Generate every possible pair (one from each array), compute their sums, sort all pairs by sum, and take the top K — O(n×m log(n×m)), generating far more pairs than needed if K is small.

Why We Optimize NEW

Sort both arrays first (so the largest possible sums are 'near the top' of each), then use a **Max-Heap** seeded with a few promising initial candidate pairs (e.g. pairing the largest elements from each array), extracting the current best sum and pushing its 'neighbouring' unexplored pairs (like decrementing one index) back into the heap, using a visited-set to avoid processing the same pair twice — repeating this K times. This explores only the pairs that could plausibly be in the top K, guided by the heap always expanding from the currently-best-known candidate, achieving roughly O(K log K) instead of generating and sorting all n×m pairs upfront.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find k maximum sum combinations from two arrays.

Visual Diagram

```
A = [1, 4, 2], B = [3, 2, 5], k = 3
       All combinations:
       1+5=6, 1+3=4, 1+2=3
       4+5=9, 4+3=7, 4+2=6
       2+5=7, 2+3=5, 2+2=4
       Top 3: [9,7,7]

       Using Max Heap:
```

```
Sort: A=[4,2,1], B=[5,3,2]
Start: (4+5, 0, 0) → 9
Add: (4+3,0,1), (2+5,1,0)
Pop: 7 (4+3) → add (4+2,0,2)
Pop: 7 (2+5) → add (2+3,1,1)
Result: [9,7,7]
```

Solution Approach

Concept: Max heap with visited set.

Algorithm

```
Algorithm MAX_SUM_COMBINATION(A, B, k):
           Sort A descending, B descending
           maxHeap = PriorityQueue(max)
           visited = HashSet()

           maxHeap.add((A[0]+B[0], 0, 0))
           visited.add((0,0))
           result = []

           While k-- > 0:
               (sum, i, j) = maxHeap.poll()
               result.add(sum)

                 If i+1 < n and (i+1,j) not visited:
                     maxHeap.add((A[i+1]+B[j], i+1, j))
                     visited.add((i+1,j))
                 If j+1 < n and (i,j+1) not visited:
                     maxHeap.add((A[i]+B[j+1], i, j+1))
                     visited.add((i,j+1))

           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> maxSumCombinations(int[] A, int[] B, int k) {
               Arrays.sort(A);
               Arrays.sort(B);
               int n = A.length;
               List<Integer> result = new ArrayList<>();

                 PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[0]
- a[0]);
                 Set<String> visited = new HashSet<>();
                 maxHeap.add(new int[]{A[n-1] + B[n-1], n-1, n-1});

               visited.add((n-1) + "," + (n-1));
```

```
            while (k-- > 0 && !maxHeap.isEmpty()) {
                int[] top = maxHeap.poll();
                int sum = top[0], i = top[1], j = top[2];
                result.add(sum);

                String key1 = (i-1) + "," + j;
                if (i > 0 && !visited.contains(key1)) {
                    maxHeap.add(new int[]{A[i-1] + B[j], i-1, j});
                    visited.add(key1);
                }

                String key2 = i + "," + (j-1);
                if (j > 0 && !visited.contains(key2)) {
                    maxHeap.add(new int[]{A[i] + B[j-1], i, j-1});
                    visited.add(key2);
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(k log k) | Space Complexity: O(k)

226. Find Median from Data Stream

Understanding the Problem NEW

Given a continuous stream of numbers, maintain the ability to report the current median (the middle value, or average of the two middle values if the count is even) at any point as new numbers arrive.

Brute Force Approach NEW

Keep all numbers seen so far in a list, and whenever asked for the median, sort the whole list and read off the middle — O(n log n) per query, which is far too slow if the median is queried frequently as the stream grows.

Why We Optimize NEW

Maintain **two heaps simultaneously**: a Max-Heap holding the smaller half of the numbers seen so far, and a Min-Heap holding the larger half — kept balanced in size (differing by at most 1). The median is then always either the root of whichever heap has more elements, or the average of both roots if they're equal in size. Every new number is inserted into the appropriate heap (based on comparison with the current roots) and the two heaps are rebalanced if needed (moving the root from one to the other) — all in O(log n) per insertion, with the median retrievable in O(1) at any time. This 'two balanced heaps, split at the median' structure is the classic solution for maintaining a running median efficiently in a streaming context.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find median of a stream of numbers.

Visual Diagram

```
Stream: [1, 2, 3, 4, 5]

       After 1: [1] → median=1
       After 2: [1,2] → median=(1+2)/2=1.5
       After 3: [1,2,3] → median=2
       After 4: [1,2,3,4] → median=(2+3)/2=2.5
       After 5: [1,2,3,4,5] → median=3
```

```
Data Structure:
Max Heap (lower half): [1,2]
Min Heap (upper half): [3,4,5]

Add 6:
  maxHeap: [1,2], minHeap: [3,4,5,6]
  Balance: maxHeap: [1,2,3], minHeap: [4,5,6]
  Median: (3+4)/2=3.5
```

Solution Approach

Concept: Two heaps (max for lower, min for upper).

Algorithm

```
Algorithm MEDIAN_STREAM:
           maxHeap = PriorityQueue(max) // lower half
           minHeap = PriorityQueue(min) // upper half

           addNum(num):
               If maxHeap.isEmpty() or num <= maxHeap.peek():
                   maxHeap.add(num)
               Else:
                   minHeap.add(num)

                 // Balance
                 If maxHeap.size() > minHeap.size() + 1:
                     minHeap.add(maxHeap.poll())
                 Else If minHeap.size() > maxHeap.size():
                     maxHeap.add(minHeap.poll())

           findMedian():
               If maxHeap.size() > minHeap.size():
                   Return maxHeap.peek()
               Else:
                   Return (maxHeap.peek() + minHeap.peek()) / 2.0
```

Java Implementation

```
class MedianFinder {
           private PriorityQueue<Integer> maxHeap;
           private PriorityQueue<Integer> minHeap;

           public MedianFinder() {
               maxHeap = new PriorityQueue<>(Collections.reverseOrder());
               minHeap = new PriorityQueue<>();
           }

           public void addNum(int num) {
```

```
            if (maxHeap.isEmpty() || num <= maxHeap.peek()) {
                maxHeap.add(num);
            } else {
                minHeap.add(num);
            }

            if (maxHeap.size() > minHeap.size() + 1) {
                minHeap.add(maxHeap.poll());
            } else if (minHeap.size() > maxHeap.size()) {
                maxHeap.add(minHeap.poll());
            }
        }

        public double findMedian() {
            if (maxHeap.size() > minHeap.size()) {
                return maxHeap.peek();
            } else {
                return (maxHeap.peek() + minHeap.peek()) / 2.0;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(n)

227. K Most Frequent Elements

Understanding the Problem NEW

Find the K most frequently occurring elements in an array. E.g. `[1,1,1,2,2,3]`, K=2 → `[1,2]` (1 appears 3 times, 2 appears twice).

Brute Force Approach NEW

Count frequencies with a hashmap, then sort all distinct elements by frequency and take the top K — O(n log n) for the sort, doing more ordering work than necessary if you only need the top K, not a full ranking of everyone.

Why We Optimize NEW

Count frequencies with a hashmap (O(n)), then maintain a **Min-Heap of size K** keyed by frequency (exactly the same 'keep only the K best, discard the rest' pattern as problem 215's Kth Largest Element) — achieving O(n log K) instead of O(n log n) for the ranking step. (An alternative **bucket sort** approach achieves O(n) instead: since frequency can range from 1 to n, create n buckets indexed by frequency, place each distinct element into its frequency's bucket, then read off the top K by scanning buckets from highest frequency down — avoiding comparison-based sorting/heap operations entirely by exploiting the bounded range of possible frequency values, similar in spirit to how the Sieve of Eratosthenes avoids per-number primality checks by using array-indexed buckets.)

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find k most frequent elements in an array.

Visual Diagram

```
Input: [1,1,1,2,2,3], k=2
       Output: [1,2]

       Frequency: 1:3, 2:2, 3:1

       Method 1: Min Heap
       Build frequency map
```

```
Add entries to min heap of size k
Result: [1,2]

Method 2: Bucket Sort
Frequency: [3,2,1]
Buckets:
0: []
1: [3]
2: [2]
3: [1]
Collect k elements: [1,2]
```

Solution Approach

Concept: Min heap or bucket sort.

Algorithm (Min Heap)

Algorithm K_FREQUENT(arr, k):

freq = new HashMap()

For num in arr: freq[num]++

minHeap = PriorityQueue(comparator by frequency)

For entry in freq:

minHeap.add(entry)

If minHeap.size() > k:

minHeap.poll()

Return minHeap entries

Java Implementation (Min Heap)

class Solution {

public int[] topKFrequent(int[] nums, int k) {

Map<Integer, Integer> freq = new HashMap<>();

for (int num : nums) {

freq.put(num, freq.getOrDefault(num, 0) + 1);

}

PriorityQueue<Map.Entry<Integer, Integer>> minHeap =

```
new PriorityQueue<>((a, b) -> a.getValue() - b.getValue());
for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
minHeap.add(entry);
if (minHeap.size() > k) {
minHeap.poll();
}
}
int[] result = new int[k];
for (int i = 0; i < k; i++) {
result[i] = minHeap.poll().getKey();
}
return result;
}
}
```

Java Implementation (Bucket Sort) - O(n)

```
class Solution {
public int[] topKFrequent(int[] nums, int k) {
Map<Integer, Integer> freq = new HashMap<>();
for (int num : nums) {
freq.put(num, freq.getOrDefault(num, 0) + 1);
}
List<Integer>[] buckets = new List[nums.length + 1];
for (int key : freq.keySet()) {
int count = freq.get(key);
if (buckets[count] == null) {
buckets[count] = new ArrayList<>();
```

```
}
buckets[count].add(key);
}
int[] result = new int[k];
int index = 0;
for (int i = buckets.length - 1; i >= 0 && index < k; i--) {
if (buckets[i] != null) {
for (int num : buckets[i]) {
result[index++] = num;
if (index == k) break;
}
}
}
return result;
}
}
```

Time & Space Complexity

Time Complexity: O(n log k) or O(n) | Space Complexity: O(n)

Heap Summary

Problem	Difficulty	Time	Space	Key Technique
Heap Basics	Learning	O(log n)	O(n)	Array-based heap
Min Heap Check	Learning	O(n)	O(1)	Property validation
Min to Max Heap	Learning	O(n)	O(log n)	Heapify all
Kth Largest	Medium	O(n log k)	O(k)	Min heap size k
Kth Smallest	Medium	O(n log k)	O(k)	Max heap size k
Sort K-Sorted	Medium	O(n log k)	O(k)	Min heap sliding
Merge K Lists	Medium	O(n log k)	O(k)	Heap with nodes
Replace with Rank	Medium	O(n log n)	O(n)	Sorting
Task Scheduler	Medium	O(n)	O(1)	Greedy + math
Hands of Straights	Medium	O(n log n)	O(n)	Heap + map
Design Twitter	Hard	O(n log k)	O(n)	Heap + maps
Connect Ropes	Hard	O(n log n)	O(n)	Min heap
Kth Largest Stream	Hard	O(log k)	O(k)	Min heap
Max Sum Combination	Hard	O(k log k)	O(k)	Heap + visited
Median Stream	Hard	O(log n)	O(n)	Two heaps
K Frequent Elements	Hard	O(n log k)	O(n)	Min heap or bucket